

PO–PSO Mapping Table :

PO	Related PSO	Rationale(one-line)
PO1	PSO1,PSO2	Applies engineering fundamentals to design secure transaction logic.
PO2	PSO3	Analyzes transaction failure modes and consistency issues.
PO3	PSO1,PSO2	Designs wallet modules and gateway adapters.
PO4	PSO3	Conducts tests and failure investigations for transactions.
PO5	PSO1,PSO2	Uses modern tools (APIs, simulators) to build integrations.
PO6	PSO1	Considers social impact of secure payment handling.
PO7	-----	(Low direct relevance)
PO8	PSO1	Ensures ethical handling of user data and consent.
PO9	PSO1,PSO2	Team collaboration to build distributed services.
PO10	PSO1,PSO2	Document APIs and produce user/admin manuals.
PO11	PSO3	Manage project scope, deployment and cost of integrations.
PO12	PSO1,PSO2	Encourages continuing study of fintech and security.

PO Justification :

- ☐ **PO1:** Engineering knowledge — Apply knowledge of mathematics, science, and engineering.
- ☐ **PO2:** Problem analysis — Identify, formulate, and analyze complex engineering problems.
- ☐ **PO3:** Design/development of solutions — Design systems to meet desired needs.
- ☐ **PO4:** Conduct investigations of complex problems.
- ☐ **PO5:** Modern tool usage — Create, select and apply appropriate techniques, resources, and modern engineering tools.
- ☐ **PO6:** The engineer and society — Apply reasoning informed by contextual knowledge.
- ☐ **PO7:** Environment and sustainability.
- ☐ **PO8:** Ethics.
- ☐ **PO9:** Individual and team work.
- ☐ **PO10:** Communication.
- ☐ **PO11:** Project management and finance.
- ☐ **PO12:** Lifelong learning.

PSO Justification :

- **PSO1:** Design and implement secure payment systems and transaction processing flows.
- **PSO2:** Integrate backend services with external payment gateways and simulate third-party interactions.
- **PSO3:** Analyze performance, reliability and consistency of distributed transaction systems.

Introduction :

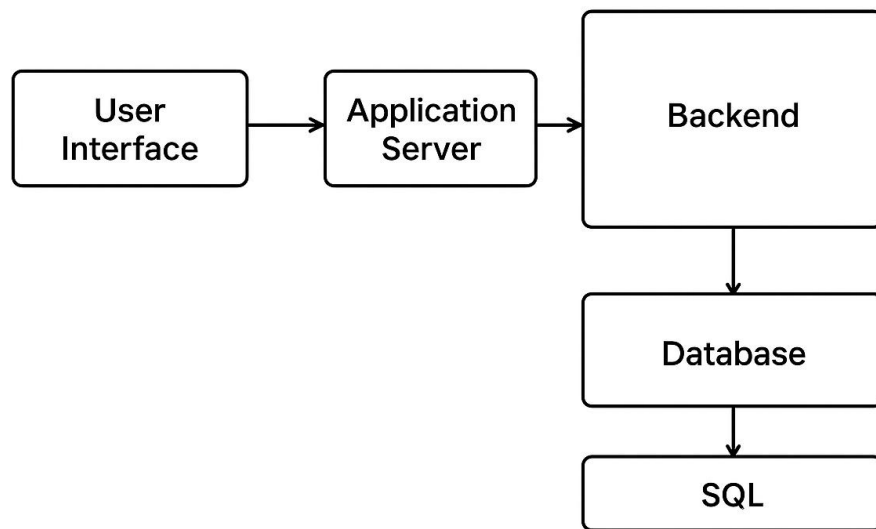
Digital Payment Wallet Simulator is an academic project that simulates the core features of a mobile/web wallet: user registration and authentication, wallet top-up, peer-to-peer transfers, merchant payments, transaction history, basic fraud checks, and admin reporting. The simulator is implemented as a modular system with a frontend (mobile/web UI), backend services (authentication, wallet, transaction processing), persistent storage (relational DB), and integration stubs for external payment gateways. The goal is to demonstrate secure transaction flows, data integrity, and basic performance under typical loads.

Project Distribution :

Module / Role	Responsibilities	Suggested People
Project Lead	Project planning, integration testing, final demo	1
Backend Lead	API design, services (Auth, Wallet, Transactions)	1-2
Frontend Lead	Mobile/Web UI, client-side validation	1-2
Database/DevOps	Schema design, migrations, Docker, deployment	1
QA & Testing	Unit/integration tests, load tests, test reports	1
Security & Compliance	Threat modelling, secure coding review	1
Documentation	User manual, API docs, report writeup	1

System Architecture :

System Architecture



Coding :

```
package binsu;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import java.sql.*;
```

```
public class DigitalWallet extends JFrame {
```

```
    JTextField txtUser, txtAmount;
```

```
    JPasswordField txtPin;
```

```
JLabel lblBalance;
```

```
Connection con;
```

```
public DigitalWallet() {
```

```
    // CONNECT TO DATABASE
```

```
    try {
```

```
        Class.forName("com.mysql.cj.jdbc.Driver");
```

```
        con = DriverManager.getConnection(
```

```
            "jdbc:mysql://localhost:3306/walletdb",
```

```
            "root",
```

```
            "Saicharan@06" // <-- CHANGE THIS
```

```
        );
```

```
    } catch (Exception e) {
```

```
        JOptionPane.showMessageDialog(this, "DB Connection Failed!");
```

```
    }
```

```
    // GUI SETTINGS
```

```
    setTitle("Digital Payment Wallet Simulator");
```

```
    setSize(500, 400);
```

```
    setLayout(null);
```

```
    setDefaultCloseOperation(EXIT_ON_CLOSE);
```

```
    JLabel l1 = new JLabel("Username:");
```

```
    l1.setBounds(40, 30, 150, 30);
```

```
    add(l1);
```

```
    txtUser = new JTextField();
```

```
    txtUser.setBounds(160, 30, 200, 30);
```

```
add(txtUser);
```

```
JLabel l2 = new JLabel("PIN:");
```

```
l2.setBounds(40, 80, 150, 30);
```

```
add(l2);
```

```
txtPin = new JPasswordField();
```

```
txtPin.setBounds(160, 80, 200, 30);
```

```
add(txtPin);
```

```
JButton btnLogin = new JButton("Login");
```

```
btnLogin.setBounds(180, 130, 100, 30);
```

```
add(btnLogin);
```

```
lblBalance = new JLabel("Balance: ₹0");
```

```
lblBalance.setBounds(40, 180, 300, 30);
```

```
lblBalance.setFont(new Font("Arial", Font.BOLD, 16));
```

```
add(lblBalance);
```

```
JLabel l3 = new JLabel("Amount:");
```

```
l3.setBounds(40, 230, 150, 30);
```

```
add(l3);
```

```
txtAmount = new JTextField();
```

```
txtAmount.setBounds(160, 230, 200, 30);
```

```
add(txtAmount);
```

```
JButton btnDeposit = new JButton("Deposit");
```

```
btnDeposit.setBounds(60, 280, 120, 30);
```

```
add(btnDeposit);
```

```
JButton btnWithdraw = new JButton("Withdraw");
btnWithdraw.setBounds(220, 280, 120, 30);
add(btnWithdraw);

// ACTION: LOGIN
btnLogin.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        loginUser();
    }
});

// ACTION: DEPOSIT
btnDeposit.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        deposit();
    }
});

// ACTION: WITHDRAW
btnWithdraw.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        withdraw();
    }
});
}

// LOGIN FUNCTION
void loginUser() {
    try {
```

```
String sql = "SELECT balance FROM users WHERE username=? AND pin=?";
```

```
PreparedStatement ps = con.prepareStatement(sql);
```

```
ps.setString(1, txtUser.getText());
```

```
ps.setString(2, String.valueOf(txtPin.getPassword()));
```

```
ResultSet rs = ps.executeQuery();
```

```
if (rs.next()) {
```

```
    lblBalance.setText("Balance: ₹" + rs.getDouble(1));
```

```
    JOptionPane.showMessageDialog(this, "Login Successful!");
```

```
} else {
```

```
    JOptionPane.showMessageDialog(this, "Invalid Username or PIN!");
```

```
}
```

```
} catch (Exception e) {
```

```
    JOptionPane.showMessageDialog(this, "Error logging in!");
```

```
}
```

```
}
```

```
// DEPOSIT FUNCTION
```

```
void deposit() {
```

```
    try {
```

```
        double amount = Double.parseDouble(txtAmount.getText());
```

```
String sql = "UPDATE users SET balance = balance + ? WHERE username=? AND  
pin=?";
```

```
PreparedStatement ps = con.prepareStatement(sql);
```

```
ps.setDouble(1, amount);
```

```
ps.setString(2, txtUser.getText());
```

```
ps.setString(3, String.valueOf(txtPin.getPassword()));
```

```
int rows = ps.executeUpdate();
```

```

        if (rows > 0) {
            JOptionPane.showMessageDialog(this, "Amount Deposited!");
            loginUser();
        } else {
            JOptionPane.showMessageDialog(this, "Login first!");
        }

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Enter valid amount!");
    }
}

// WITHDRAW FUNCTION
void withdraw() {
    try {
        double amount = Double.parseDouble(txtAmount.getText());

        String sql = "UPDATE users SET balance = balance - ? WHERE username=? AND pin=? AND balance >= ?";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setDouble(1, amount);
        ps.setString(2, txtUser.getText());
        ps.setString(3, String.valueOf(txtPin.getPassword()));
        ps.setDouble(4, amount);

        int rows = ps.executeUpdate();
        if (rows > 0) {
            JOptionPane.showMessageDialog(this, "Amount Withdrawn!");
            loginUser();
        } else {
            JOptionPane.showMessageDialog(this, "Insufficient Balance or Login Required!");
        }
    }
}

```

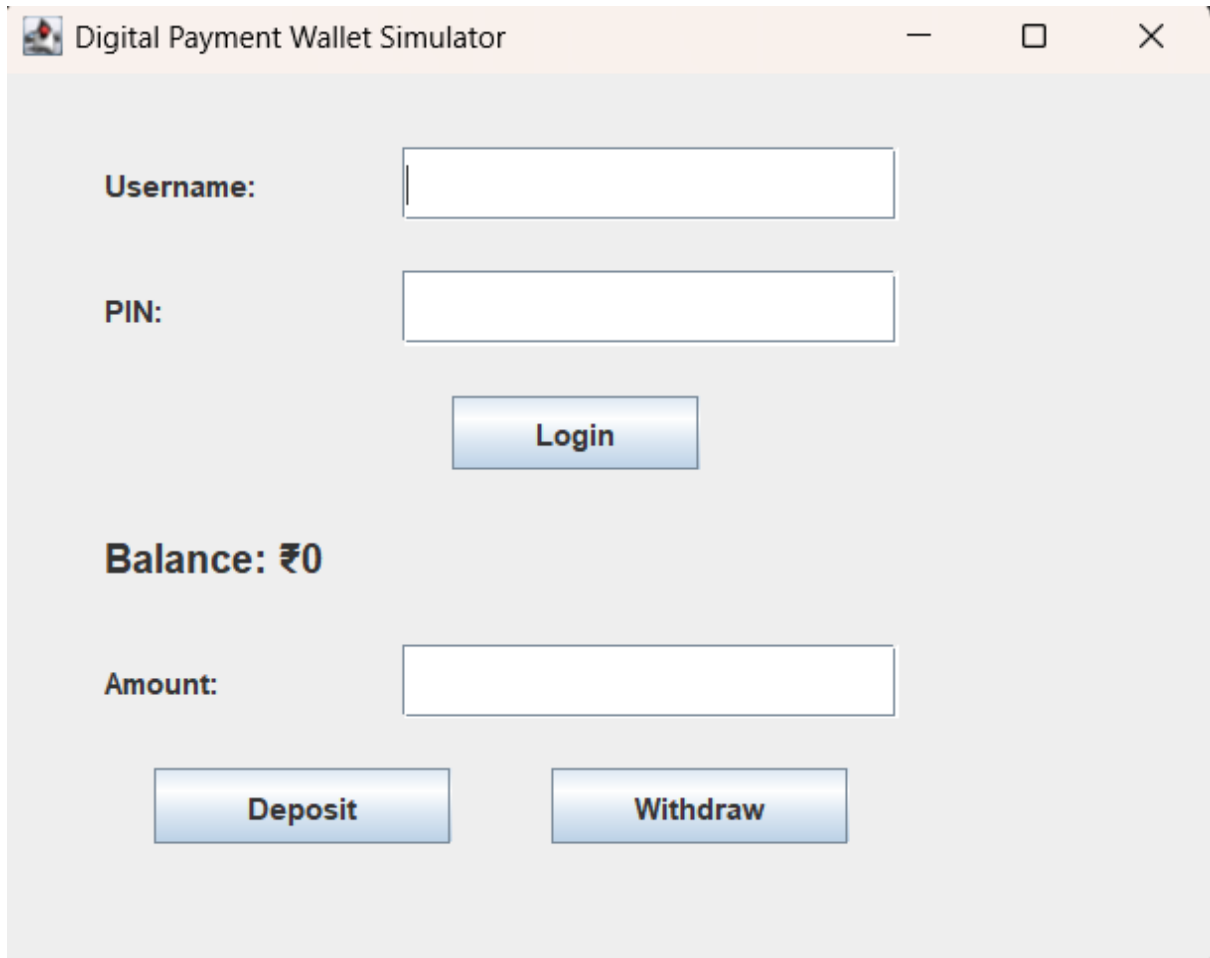
```
    }

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Enter valid amount!");
    }
}

// MAIN METHOD
public static void main(String[] args) {
    new DigitalWallet().setVisible(true);
}
}
```

SCREENSHOTS :

FRONTEND :



The screenshot displays the frontend of a 'Digital Payment Wallet Simulator'. The application window has a title bar with the text 'Digital Payment Wallet Simulator' and standard window control buttons (minimize, maximize, close). The main interface is light gray and contains the following elements:

- Username:** A text label followed by a white rectangular input field.
- PIN:** A text label followed by a white rectangular input field.
- Login:** A blue rectangular button with white text, positioned below the PIN field.
- Balance: ₹0**: A text label indicating the current wallet balance.
- Amount:** A text label followed by a white rectangular input field.
- Deposit:** A blue rectangular button with white text, located at the bottom left.
- Withdraw:** A blue rectangular button with white text, located at the bottom right.

Digital Payment Wallet Simulator

Username:

PIN:

Balance:

Amount:

Message

 Login Successful!

Digital Payment Wallet Simulator

Username:

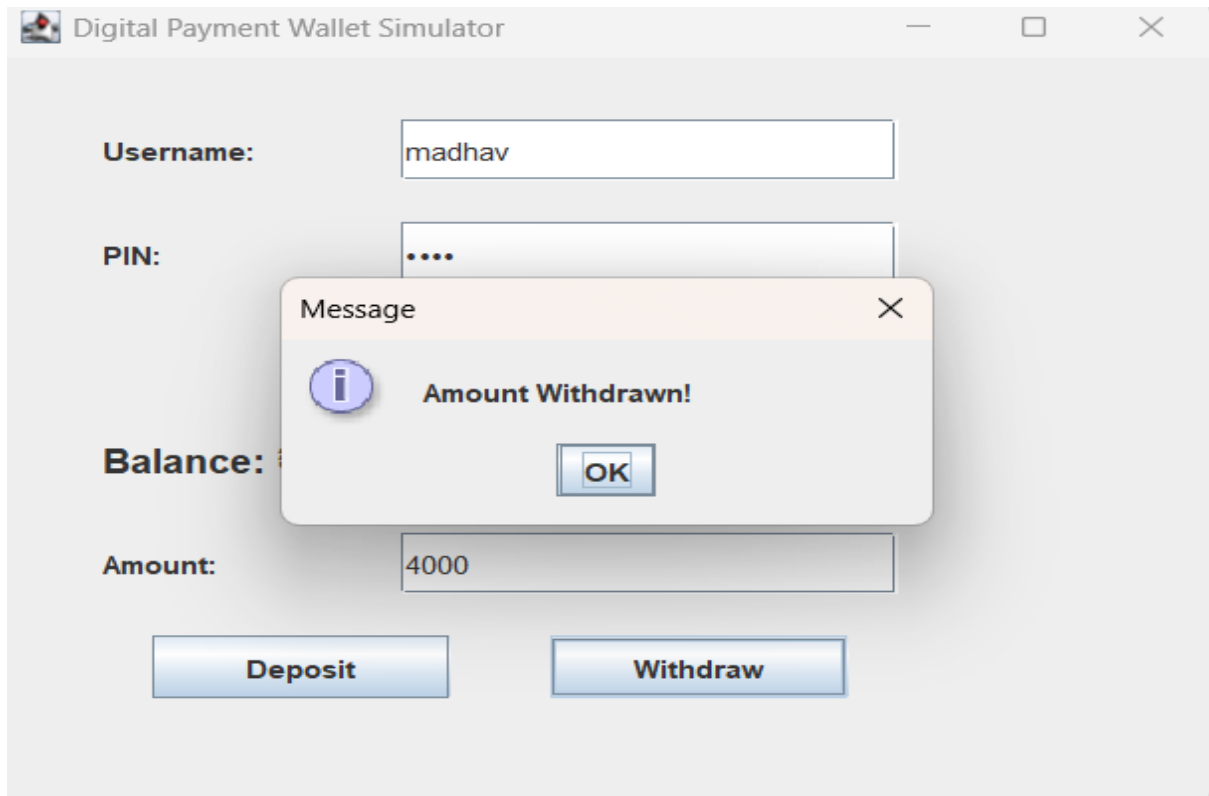
PIN:

Balance:

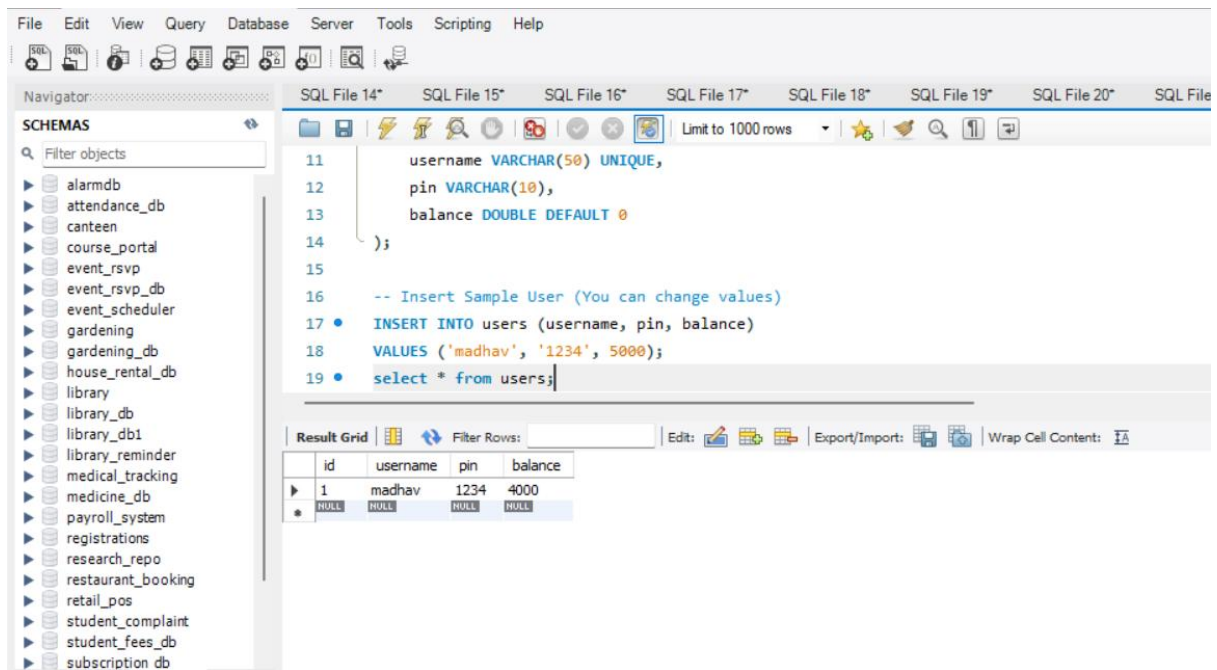
Amount:

Message

 Amount Deposited!



BACKEND :



CONCLUSION :

The *Digital Payment Wallet Simulator* successfully demonstrates how modern digital payment systems work in a secure and user-friendly manner. Through features like user registration, login authentication, balance checking, money deposit, withdrawal, and transaction history, the system simulates real-world wallet functionality. With the help of Java, JDBC, and SQL database integration, the project provides a practical understanding of backend connectivity, data storage, and graphical user interface implementation. Overall, the project enhances knowledge in Java programming, database management, and GUI-based application development, making it a valuable learning experience for understanding digital financial systems.

Future Work :

1. Add Transaction Security

Implement encryption, OTP verification, and secure password hashing for real-world safety.

2. Integrate QR-Code Payments

Allow users to scan and pay using QR codes similar to UPI apps.

3. Add Wallet-to-Wallet Transfer

Enable users to send money to other registered wallet users.

4. Add Notifications

Provide SMS/Email alerts for each transaction.

5. Add Data Analytics

Include monthly spending reports and charts.

6. Add Mobile App Support

Convert the project into an Android application using Java or Kotlin.

7. Add Admin Dashboard

Allow an admin to monitor users, transactions, and system activity.

8. Cloud Database Integration

Move the data storage from local MySQL to cloud servers for better scalability.